

Denau Hermanus

ST10497747

PROG5121

Assignment PART 1

[denauhermanus881-hash/ST10497747-PROG5121-POE-Part1](#)

The screenshot shows an IDE with the `MainApp.java` file open. The code defines a `MainApp` class with a `main` method that prompts the user for login details and registers a new user if the conditions are met.

```
src > J MainApp.java > Language Support for Java(TM) by Red Hat > MainApp
3 public class MainApp {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
6.3         String pinInput = sc.nextLine();
6.4
6.5         User user = new User();
6.6         user.loginUser(sc.nextLine(), sc.nextLine());
6.7         System.out.println(user.returnLoginStatus(isLoggedIn));
6.8     }
6.9
6.10    sc.close();
71 }
```

The terminal output shows the execution of the program:

```
PS C:\Users\Student\Desktop\PROGS121_PART1> cd 'C:\Users\Student\Desktop\PROGS121_PART1'; & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '%CD%\src\MainApp.java'
Enter Last Name: Hermanus!!!
Enter Username: den38
Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.
Enter Username: denau_38
Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.
Enter Username: den_3
Username successfully captured.
Enter Password: 38baby
Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character.
Enter Password: 38baby!#
Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character.
Enter Password: 38baby!#
Password successfully captured.
Enter Cell Number: 0769856209
Cell number is incorrectly formatted or does not contain an international code; please correct the number and try again.
Enter Cell Number: +279856309
Cell phone number successfully added.
The two above conditions have been met, and the user has been registered successfully.

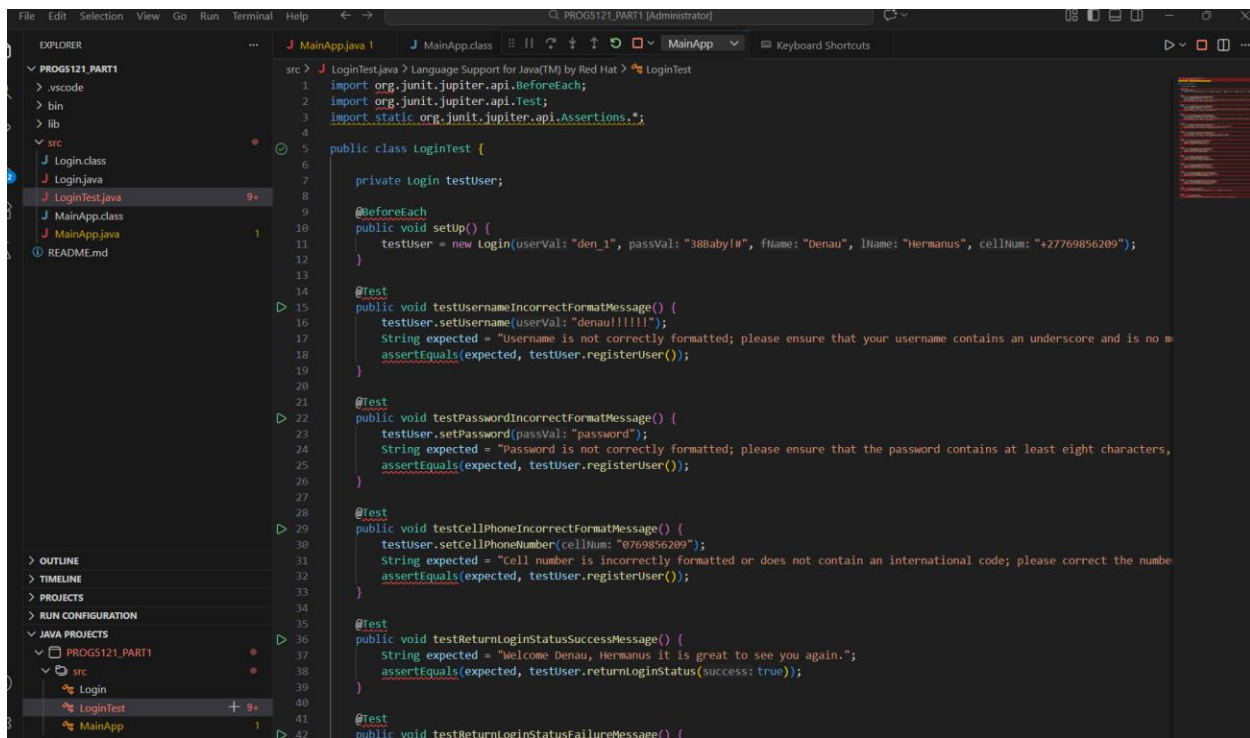
Login
Enter Username: den_1
Enter Password: 38baby!#
Username or password incorrect, please try again.
Enter Username:
```

The screenshot shows an IDE with the `Login.java` file open. The code defines a `Login` class with private attributes for user, password, first name, last name, and cell number, and public methods for login, getters, and setters.

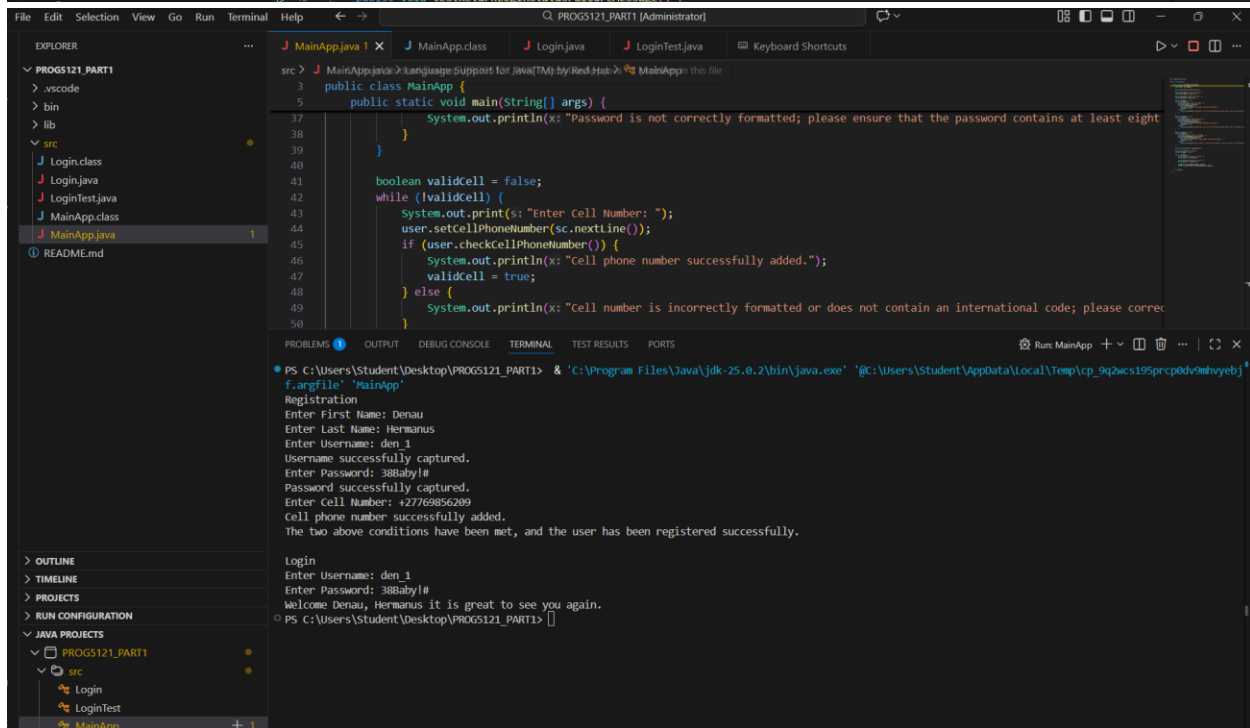
```
src > J Login.java > Language Support for Java(TM) by Red Hat > Login
1 public class Login {
2
3     private String userVal;
4     private String passVal;
5     private String fName;
6     private String lName;
7     private String cellNum;
8
9     public Login() {
10    }
11
12    public login(String userVal, String passVal, String fName, String lName, String cellNum) {
13        this.userVal = userVal;
14        this.passVal = passVal;
15        this.fName = fName;
16        this.lName = lName;
17        this.cellNum = cellNum;
18    }
19
20    public String getUsername() {
21        return userVal;
22    }
23
24    public void setUsername(String userVal) {
25        this.userVal = userVal;
26    }
27
28    public String getPassword() {
29        return passVal;
30    }
31
32    public void setPassword(String passVal) {
33        this.passVal = passVal;
34    }
35
36    public String getFirstName() {
37        return fName;
38    }
39
40    public void setFirstName(String fName) {
41        this.fName = fName;
42    }
43 }
```

```
src > LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
5 public class LoginTest {
57
58
59
60 @Test
61 public void testPasswordMeetsComplexity() {
62     testUser.setPassword(passVal: "388Baby1#");
63     assertTrue(testUser.checkPasswordComplexity());
64
65
66 @Test
67 public void testPasswordDoesNotMeetComplexity() {
68     testUser.setPassword(passVal: "password");
69     assertFalse(testUser.checkPasswordComplexity());
70
71
72 @Test
73 public void testCellPhoneNumberCorrectlyFormatted() {
74     testUser.setCellPhoneNumber(cellNum: "+27769856209");
75     assertTrue(testUser.checkCellPhoneNumber());
76
77
78 @Test
79 public void testCellPhoneNumberIncorrectlyFormatted() {
80     testUser.setCellPhoneNumber(cellNum: "0769856209");
81     assertFalse(testUser.checkCellPhoneNumber());
82
83
84 @Test
85 public void testLoginSuccessful() {
86     assertTrue(testUser.loginUser(inputUser: "den_1", inputPass: "388Baby1#"));
87
88
89 @Test
90 public void testLoginFailed() {
91     assertFalse(testUser.loginUser(inputUser: "den_1", inputPass: "wrongPassword"));
92 }
```

```
src > LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
36 public class LoginTest {
37     public void testReturnLoginStatusSuccessMessage() {
38         String expected = "Welcome Denau, Hermanus it is great to see you again.";
39         assertEquals(expected, testUser.returnLoginStatus(success: true));
40
41
42 @Test
43 public void testReturnLoginStatusFailureMessage() {
44     String expected = "Username or password incorrect, please try again.";
45     assertEquals(expected, testUser.returnLoginStatus(success: false));
46
47
48 @Test
49 public void testUsernameCorrectlyFormatted() {
50     testUser.setUsername(userVal: "den_1");
51     assertTrue(testUser.checkUserName());
52
53
54 @Test
55 public void testUsernameIncorrectlyFormatted() {
56     testUser.setUsername(userVal: "denau1!!!!");
57     assertFalse(testUser.checkUserName());
58
59
60 @Test
61 public void testPasswordMeetsComplexity() {
62     testUser.setPassword(passVal: "388Baby1#");
63     assertTrue(testUser.checkPasswordComplexity());
64
65
66 @Test
67 public void testPasswordDoesNotMeetComplexity() {
68     testUser.setPassword(passVal: "password");
69     assertFalse(testUser.checkPasswordComplexity());
70
71
72 @Test
73 public void testCellPhoneNumberCorrectlyFormatted() {
74     testUser.setCellPhoneNumber(cellNum: "+27769856209");
75     assertTrue(testUser.checkCellPhoneNumber());
76 }
```



```
src > J LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 public class LoginTest {
6
7     private Login testUser;
8
9     @BeforeEach
10    public void setUp() {
11        testUser = new Login(userVal: "den_1", passVal: "388Baby!#", fName: "Denau", lName: "Hermanus", cellNum: "+27769856209");
12    }
13
14    @Test
15    public void testUsernameIncorrectFormatMessage() {
16        testUser.setUsername(userVal: "denau!!!!!!");
17        String expected = "Username is not correctly formatted; please ensure that your username contains an underscore and is no
18        assertEquals(expected, testUser.registerUser());
19    }
20
21    @Test
22    public void testPasswordIncorrectFormatMessage() {
23        testUser.setPassword(passVal: "password");
24        String expected = "Password is not correctly formatted; please ensure that the password contains at least eight characters,
25        assertEquals(expected, testUser.registerUser());
26    }
27
28    @Test
29    public void testCellPhoneIncorrectFormatMessage() {
30        testUser.setCellPhoneNumber(cellNum: "0769856209");
31        String expected = "Cell number is incorrectly formatted or does not contain an international code; please correct the numbe
32        assertEquals(expected, testUser.registerUser());
33    }
34
35    @Test
36    public void testReturnLoginStatusSuccessMessage() {
37        String expected = "Welcome Denau, Hermanus it is great to see you again.";
38        assertEquals(expected, testUser.returnLoginStatus(success: true));
39    }
40
41    @Test
42    public void testReturnLoginStatusFailureMessage() {
```



```
src > J MainApp.java > Language Support for Java(TM) by Red Hat > MainApp this file
3 public class MainApp {
4     public static void main(String[] args) {
5
6         System.out.println(x: "Password is not correctly formatted; please ensure that the password contains at least eight
7     }
8 }
9
10 boolean validCell = false;
11 while (!validCell) {
12     System.out.print(s: "Enter Cell Number: ");
13     user.setCellPhoneNumber(sc.nextLine());
14     if (user.checkCellPhoneNumber()) {
15         System.out.println(x: "Cell phone number successfully added.");
16         validCell = true;
17     } else {
18         System.out.println(x: "Cell number is incorrectly formatted or does not contain an international code; please correct
19     }
20 }
21
22 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
23
24 PS C:\Users\Student\Desktop\PROGS121_PART1> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '@C:\Users\Student\AppData\Local\Temp\cp_9q2wcs195prcpddy9mhvyebl'
f:\argfile' 'MainApp'
Registration
Enter First Name: Denau
Enter Last Name: Hermanus
Enter Username: den_1
Username successfully captured.
Enter Password: 388Baby!#
Password successfully captured.
Enter Cell Number: +27769856209
Cell phone number successfully added.
The two above conditions have been met, and the user has been registered successfully.
Login
Enter Username: den_1
Enter Password: 388Baby!#
Welcome Denau, Hermanus it is great to see you again.
PS C:\Users\Student\Desktop\PROGS121_PART1> []
```

The screenshot shows an IDE with the `MainApp.java` file open. The code defines a `MainApp` class with a `main` method that prompts the user for login details. The terminal output shows the program running and accepting user input.

```
src > J MainApp.java > Language Support for Java(TM) by Red Hat > MainApp
3 public class MainApp {
5     public static void main(String[] args) {
6         String pinInput = sc.nextLine();
63
64
65         isLoggedIn = user.loginUser(uInput, pInput);
66         System.out.println(user.returnLoginStatus(isLoggedIn));
67     }
68
69     sc.close();
70 }
71 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

PS C:\Users\Student\Desktop\PROG5121_PART1> c:\cd 'C:\Users\Student\Desktop\PROG5121_PART1'; & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '@C:\Users\Student\AppData\Local\Temp\cp_5q2wcs195prcp8dv9mhyebjf\argfile' 'MainApp'

Enter Last Name: Hermanus!!!
Enter Username: den38
Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.
Enter Username: denau_38
Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.
Enter Username: den_3
Username successfully captured.
Enter Password: 38baby
Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character.
Enter Password: 38baby!#
Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character.
Enter Password: 38babyv!
Password successfully captured.
Enter Cell Number: 0769856209
Cell number is incorrectly formatted or does not contain an international code; please correct the number and try again.
Enter Cell Number: +279856309
Cell phone number successfully added.
The two above conditions have been met, and the user has been registered successfully.

Login
Enter Username: den_1
Enter Password: 38babyv!#
Username or password incorrect, please try again.
Enter Username:

The screenshot shows an IDE with the `Login.java` file open. The code defines a `Login` class with private attributes for user details and public methods for login, getters, and setters.

```
src > J Login.java > Language Support for Java(TM) by Red Hat > Login
1 public class Login {
2
3     private String userVal;
4     private String passVal;
5     private String fName;
6     private String lName;
7     private String cellNum;
8
9     public Login() {
10
11
12     public Login(String userVal, String passVal, String fName, String lName, String cellNum) {
13         this.userVal = userVal;
14         this.passVal = passVal;
15         this.fName = fName;
16         this.lName = lName;
17         this.cellNum = cellNum;
18     }
19
20     public String getUsername() {
21         return userVal;
22     }
23
24     public void setUsername(String userVal) {
25         this.userVal = userVal;
26     }
27
28     public String getPassword() {
29         return passVal;
30     }
31
32     public void setPassword(String passVal) {
33         this.passVal = passVal;
34     }
35
36     public String getFirstName() {
37         return fName;
38     }
39
40     public void setFirstName(String fName) {
41         this.fName = fName;
42     }
43 }
```

```
src > J LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
5 public class LoginTest {
57
58
59
60 @Test
61 public void testPasswordMeetsComplexity() {
62     testUser.setPassword(passVal: "38Babyl#");
63     assertTrue(testUser.checkPasswordComplexity());
64
65
66 @Test
67 public void testPasswordDoesNotMeetComplexity() {
68     testUser.setPassword(passVal: "password");
69     assertFalse(testUser.checkPasswordComplexity());
70
71
72 @Test
73 public void testCellPhoneNumberCorrectlyFormatted() {
74     testUser.setCellPhoneNumber(cellNum: "+27769856209");
75     assertTrue(testUser.checkCellPhoneNumber());
76
77
78 @Test
79 public void testCellPhoneNumberIncorrectlyFormatted() {
80     testUser.setCellPhoneNumber(cellNum: "0769856209");
81     assertFalse(testUser.checkCellPhoneNumber());
82
83
84 @Test
85 public void testLoginSuccessful() {
86     assertTrue(testUser.loginUser(inputUser: "den_1", inputPass: "38Babyl#"));
87
88
89 @Test
90 public void testLoginFailed() {
91     assertFalse(testUser.loginUser(inputUser: "den_1", inputPass: "wrongPassword"));
92 }
```

```
src > J LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
5 public class LoginTest {
36 public void testReturnLoginStatusSuccessMessage() {
37     String expected = "welcome Denau, Hermanus it is great to see you again.";
38     assertEquals(expected, testUser.returnLoginStatus(success: true));
39
40
41
42 @Test
43 public void testReturnLoginStatusFailureMessage() {
44     String expected = "Username or password incorrect, please try again.";
45     assertEquals(expected, testUser.returnLoginStatus(success: false));
46
47
48 @Test
49 public void testUsernameCorrectlyFormatted() {
50     testUser.setUsername(userVal: "den_1");
51     assertTrue(testUser.checkUserName());
52
53
54 @Test
55 public void testUsernameIncorrectlyFormatted() {
56     testUser.setUsername(userVal: "denau11111");
57     assertFalse(testUser.checkUserName());
58
59
60 @Test
61 public void testPasswordMeetsComplexity() {
62     testUser.setPassword(passVal: "38Babyl#");
63     assertTrue(testUser.checkPasswordComplexity());
64
65
66 @Test
67 public void testPasswordDoesNotMeetComplexity() {
68     testUser.setPassword(passVal: "password");
69     assertFalse(testUser.checkPasswordComplexity());
70
71
72 @Test
73 public void testCellPhoneNumberCorrectlyFormatted() {
74     testUser.setCellPhoneNumber(cellNum: "+27769856209");
75     assertTrue(testUser.checkCellPhoneNumber());
76 }
```

```
src > J LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
1  import org.junit.jupiter.api.BeforeEach;
2  import org.junit.jupiter.api.Test;
3  import static org.junit.jupiter.api.Assertions.*;
4
5  public class LoginTest {
6
7      private Login testUser;
8
9      @BeforeEach
10     public void setUp() {
11         testUser = new Login(userVal: "den1", passVal: "388Baby!#", fName: "Denau", lName: "Hermanus", cellNum: "+27769856209");
12     }
13
14     @Test
15     public void testUsernameIncorrectFormatMessage() {
16         testUser.setUsername(userVal: "denau!!!!");
17         String expected = "Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.";
18         assertEquals(expected, testUser.registerUser());
19     }
20
21     @Test
22     public void testPasswordIncorrectFormatMessage() {
23         testUser.setPassword(passVal: "password");
24         String expected = "Password is not correctly formatted; please ensure that the password contains at least eight characters, a lowercase letter, an uppercase letter, a digit, and a special character.";
25         assertEquals(expected, testUser.registerUser());
26     }
27
28     @Test
29     public void testCellPhoneIncorrectFormatMessage() {
30         testUser.setCellPhoneNumber(cellNum: "0769856209");
31         String expected = "Cell number is incorrectly formatted or does not contain an international code; please correct the number to include a valid international code and a valid cell number.";
32         assertEquals(expected, testUser.registerUser());
33     }
34
35     @Test
36     public void testReturnLoginStatusSuccessMessage() {
37         String expected = "Welcome Denau, Hermanus it is great to see you again.";
38         assertEquals(expected, testUser.returnLoginStatus(success: true));
39     }
40
41     @Test
42     public void testReturnLoginStatusFailureMessage() {
43         String expected = "Invalid login details. Please check your username and password and try again.";
44         assertEquals(expected, testUser.returnLoginStatus(success: false));
45     }
46 }
```

```
src > J LoginTest.java > Language Support for Java(TM) by Red Hat > LoginTest
1  import org.junit.jupiter.api.BeforeEach;
2  import org.junit.jupiter.api.Test;
3  import static org.junit.jupiter.api.Assertions.*;
4
5  public class LoginTest {
6
7      private Login testUser;
8
9      @BeforeEach
10     public void setUp() {
11         testUser = new Login(userVal: "den1", passVal: "388Baby!#", fName: "Denau", lName: "Hermanus", cellNum: "+27769856209");
12     }
13
14     @Test
15     public void testUsernameIncorrectFormatMessage() {
16         testUser.setUsername(userVal: "denau!!!!");
17         String expected = "Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.";
18         assertEquals(expected, testUser.registerUser());
19     }
20
21     @Test
22     public void testPasswordIncorrectFormatMessage() {
23         testUser.setPassword(passVal: "password");
24         String expected = "Password is not correctly formatted; please ensure that the password contains at least eight characters, a lowercase letter, an uppercase letter, a digit, and a special character.";
25         assertEquals(expected, testUser.registerUser());
26     }
27
28     @Test
29     public void testCellPhoneIncorrectFormatMessage() {
30         testUser.setCellPhoneNumber(cellNum: "0769856209");
31         String expected = "Cell number is incorrectly formatted or does not contain an international code; please correct the number to include a valid international code and a valid cell number.";
32         assertEquals(expected, testUser.registerUser());
33     }
34
35     @Test
36     public void testReturnLoginStatusSuccessMessage() {
37         String expected = "Welcome Denau, Hermanus it is great to see you again.";
38         assertEquals(expected, testUser.returnLoginStatus(success: true));
39     }
40
41     @Test
42     public void testReturnLoginStatusFailureMessage() {
43         String expected = "Invalid login details. Please check your username and password and try again.";
44         assertEquals(expected, testUser.returnLoginStatus(success: false));
45     }
46 }
```

```
PS C:\Users\Student\Desktop\PROG5121_PART1> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '@C:\Users\Student\AppData\Local\Temp\cp_9q2wcs195prcp8dv9mhvyebjf.argfile' 'MainApp'
Registration
Enter First Name: git init
Enter Last Name: git add .
Enter Username: git commit -m "ST10497747 - Completed PROG5121 POE Part 1 Registration and Login"
Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length.
Enter Username: 
```

```
5 public class LoginTest {
6
7     private Login testUser;
8 }
9+

PROBLEMS 32 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
1 PS C:\Users\Student\Desktop\PROG5121_PART1> git branch -M main
+ CategoryInfo          : ObjectNotFound: (git:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Student\Desktop\PROG5121_PART1> git remote add origin https://github.com/denauhermanus881-h
ash/ST10497747-PROG5121-POE-Part1.git
git : The term 'git' is not recognized as the name of a cmdlet, function, script file, or operable
program. Check the spelling of the name, or if a path was included, verify that the path is correct
and try again.
At line:1 char:1
+ git remote add origin https://github.com/denauhermanus881-hash/ST1049 ...
+ ~~~~
+ CategoryInfo          : ObjectNotFound: (git:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Student\Desktop\PROG5121_PART1> git push -u origin main --force
git : The term 'git' is not recognized as the name of a cmdlet, function, script file, or operable
program. Check the spelling of the name, or if a path was included, verify that the path is correct
and try again.
At line:1 char:1
+ git push -u origin main --force
+ ~~~~
+ CategoryInfo          : ObjectNotFound: (git:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
+ 9+
1
```

```
PS C:\Users\Student\Desktop\PROG5121_PART1> git init
Initialized empty Git repository in C:/Users/Student/Desktop/PROG5121_PART1/.git/
PS C:\Users\Student\Desktop\PROG5121_PART1> git add .
PS C:\Users\Student\Desktop\PROG5121_PART1> git commit -m "ST10497747 - Completed PROG5121 POE Part 1 Registration and Login"
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Student@LabVM2049939.(none)')
PS C:\Users\Student\Desktop\PROG5121_PART1> git branch -M main
PS C:\Users\Student\Desktop\PROG5121_PART1> git remote add origin https://github.com/denauhermanus881-hash/ST10497747-PROG5121-POE-Part1.git
PS C:\Users\Student\Desktop\PROG5121_PART1> git push -u origin main --force
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/denauhermanus881-hash/ST10497747-PROG5121-POE-Part1.git'
PS C:\Users\Student\Desktop\PROG5121_PART1>
```

```
rm 'src/Login.class'
rm 'src/MainApp.class'
● PS C:\Users\Student\Desktop\PROG5121_PART1> git rm --cached bin/*.class
rm 'bin/Login.class'
rm 'bin/LoginTest.class'
rm 'bin/MainApp.class'
● PS C:\Users\Student\Desktop\PROG5121_PART1> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    bin/Login.class
        deleted:    bin/LoginTest.class
        deleted:    bin/MainApp.class
        deleted:    src/Login.class
        deleted:    src/MainApp.class

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bin/
        src/Login.class
        src/MainApp.class
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    bin/Login.class
        deleted:    bin/LoginTest.class
        deleted:    bin/MainApp.class
        deleted:    src/Login.class
        deleted:    src/MainApp.class

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        src/.gitignore
```

```

PS C:\Users\Student\Desktop\PROG5121_PART1> git add src/LoginTest.java
>> git commit -m "Add LoginTest class containing JUnit test cases for auth criteria"
nothing to commit, working tree clean
PS C:\Users\Student\Desktop\PROG5121_PART1> git add POE_Part1_Screenshots.docx
>> git commit -m "Add Word document containing application execution and unit test screenshots"
fatal: pathspec 'POE_Part1_Screenshots.docx' did not match any files
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\Student\Desktop\PROG5121_PART1> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 457 bytes | 228.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/denauhermanus881-hash/ST10497747-PROG5121-POE-Part1.git
 92a8132..dc2ada3  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Student\Desktop\PROG5121_PART1> git log --oneline
dc2ada3 (HEAD -> main, origin/main) Ignore compiled class files and build artifacts
92a8132 ST10497747 - Completed PROG5121 POE Part 1 Registration and Login
PS C:\Users\Student\Desktop\PROG5121_PART1>

PS C:\Users\Student\Desktop\PROG5121_PART1> git add src/Login.java
>> git commit -m "Refactor Login.java with inline documentation for validation methods"
 1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git add src/MainApp.java
>> git commit -m "Enhance MainApp.java console messaging and user prompt flow"
[main c28d961] Enhance MainApp.java console messaging and user prompt flow
 1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git add src/LoginTest.java
>> git commit -m "Add descriptive assertions and comments to LoginTest.java unit tests"
[main 7572091] Add descriptive assertions and comments to LoginTest.java unit tests
 1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git add README.md
>> git commit -m "Add project README with setup instructions and project overview"
[main ba0ab72] Add project README with setup instructions and project overview
 1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 4 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 1.53 KiB | 392.00 KiB/s, done.
Total 15 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), completed with 5 local objects.
To https://github.com/denauhermanus881-hash/ST10497747-PROG5121-POE-Part1.git
 dc2ada3..ba0ab72  main -> main
PS C:\Users\Student\Desktop\PROG5121 PART1>

```

```

PS C:\Users\Student\Desktop\PROG5121_PART1> git add src/LoginTest.java
>> git commit -m "Add descriptive assertions and comments to LoginTest.java unit tests"
[main 7572091] Add descriptive assertions and comments to LoginTest.java unit tests
1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git add README.md
>> git commit -m "Add project README with setup instructions and project overview"
[main ba0ab72] Add project README with setup instructions and project overview
1 file changed, 2 insertions(+)
PS C:\Users\Student\Desktop\PROG5121_PART1> git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 4 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 1.53 KiB | 392.00 KiB/s, done.
Total 15 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), completed with 5 local objects.
To https://github.com/denauhermanus881-hash/ST10497747-PROG5121-POE-Part1.git
dc2ada3..ba0ab72  main -> main
PS C:\Users\Student\Desktop\PROG5121_PART1> git log --oneline
ba0ab72 (HEAD -> main, origin/main) Add project README with setup instructions and project overview
7572091 Add descriptive assertions and comments to LoginTest.java unit tests
c28d961 Enhance MainApp.java console messaging and user prompt flow
4ea7253 Refactor Login.java with inline documentation for validation methods
dc2ada3 Ignore compiled class files and build artifacts
92a8132 ST10497747 - Completed PROG5121 POE Part 1 Registration and Login
PS C:\Users\Student\Desktop\PROG5121_PART1>

```

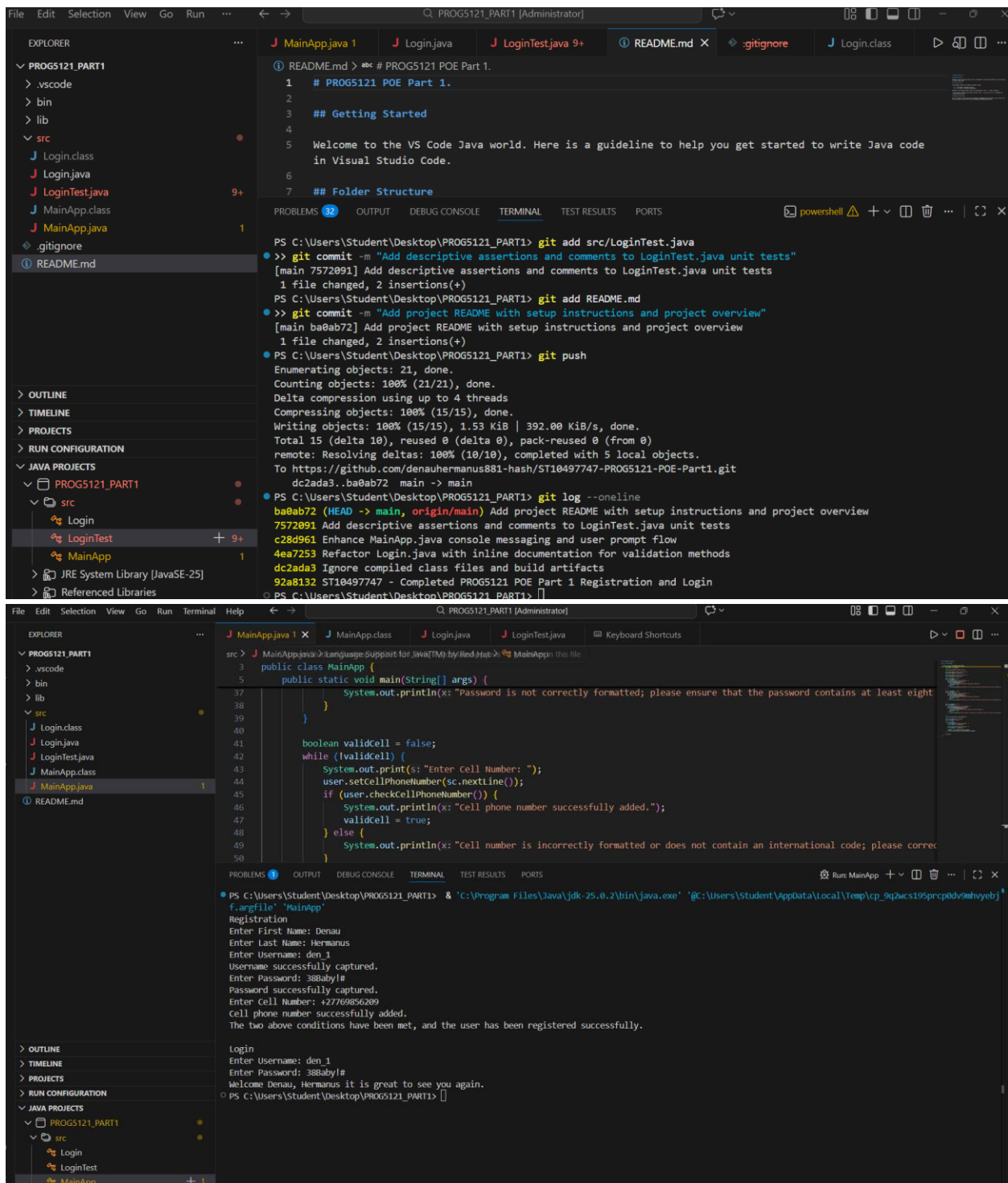
The screenshot shows the Visual Studio Code interface with the following components:

- Explorer View:** Displays the file structure of the project 'PROG5121_PART1'. The 'src' folder is expanded, showing files like 'Login.class', 'Login.java', 'LoginTest.java' (9+ changes), 'MainApp.class', and 'MainApp.java' (1 change). The 'README.md' file is also visible.
- Editor View:** Shows the content of 'README.md'. The text includes:


```

1 # PROG5121 POE Part 1.
2
3 ## Getting Started
4
5 Welcome to the VS Code Java world. Here is a guideline to help you get started to write Java code
  in Visual Studio Code.
6
7 ## Folder Structure

```
- Terminal View:** Displays the output of the Git commands executed in the PowerShell terminal, matching the text in the first block. It shows the successful execution of 'git add', 'git commit', and 'git push' commands, along with the 'git log --oneline' output.



Login.java

// Login class handling authentication logic.

```
public class Login {
```

```
    private String userVal;
```

```
    private String passVal;
```

```
    private String fName;
```

```
    private String lName;
```

```
    private String cellNum;
```

```
    public Login() {
```

```
    }
```

```
    public Login(String userVal, String passVal, String fName, String lName, String cellNum) {
```

```
        this.userVal = userVal;
```

```
        this.passVal = passVal;
```

```
        this.fName = fName;
```

```
        this.lName = lName;
```

```
        this.cellNum = cellNum;
```

```
    }
```

```
    public String getUsername() {
```

```
return userVal;  
}
```

```
public void setUsername(String userVal) {  
    this.userVal = userVal;  
}
```

```
public String getPassword() {  
    return passVal;  
}
```

```
public void setPassword(String passVal) {  
    this.passVal = passVal;  
}
```

```
public String getFirstName() {  
    return fName;  
}
```

```
public void setFirstName(String fName) {  
    this.fName = fName;  
}
```

```
public String getLastName() {  
    return lName;  
}
```

```
public void setLastName(String lName) {  
    this.lName = lName;  
}
```

```
public String getCellPhoneNumber() {  
    return cellNum;  
}
```

```
public void setCellPhoneNumber(String cellNum) {  
    this.cellNum = cellNum;  
}
```

```
public boolean checkUserName() {  
    if (userVal == null) return false;  
    return userVal.contains("_") && userVal.length() <= 5;  
}
```

```
public boolean checkPasswordComplexity() {  
    if (passVal == null || passVal.length() < 8) {  
        return false;  
    }
```

```
    boolean upperFound = false;  
    boolean numFound = false;  
    boolean specFound = false;
```

```
for (int i = 0; i < passVal.length(); i++) {  
    char c = passVal.charAt(i);  
    if (Character.isUpperCase(c)) {  
        upperFound = true;  
    } else if (Character.isDigit(c)) {  
        numFound = true;  
    } else if (!Character.isLetterOrDigit(c)) {  
        specFound = true;  
    }  
}  
  
return upperFound && numFound && specFound;  
}
```

```
public boolean checkCellPhoneNumber() {  
    if (cellNum == null) return false;  
    return cellNum.matches("^\\+27\\d{8,10}$");  
}
```

```
public String registerUser() {  
    if (!checkUserName()) {  
        return "Username is not correctly formatted; please ensure that your username contains  
        an underscore and is no more than five characters in length.";  
    }  
  
    if (!checkPasswordComplexity()) {
```

```
return "Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character.";
```

```
}
```

```
if (!checkCellPhoneNumber()) {
```

```
    return "Cell number is incorrectly formatted or does not contain an international code; please correct the number and try again.";
```

```
}
```

```
return "The two above conditions have been met, and the user has been registered successfully.";
```

```
}
```

```
public boolean loginUser(String inputUser, String inputPass) {
```

```
    if (inputUser == null || inputPass == null) return false;
```

```
    return inputUser.equals(this.userVal) && inputPass.equals(this.passVal);
```

```
}
```

```
public String returnLoginStatus(boolean success) {
```

```
    if (success) {
```

```
        return "Welcome " + fName + ", " + lName + " it is great to see you again.";
```

```
    } else {
```

```
        return "Username or password incorrect, please try again.";
```

```
    }
```

```
}
```

```
}
```

MainApp.java

```
// Main application entry point.
```

```
import java.util.Scanner;
```

```
public class MainApp {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        Login user = new Login();
```

```
        System.out.println("Registration");
```

```
        System.out.print("Enter First Name: ");
```

```
        user.setFirstName(sc.nextLine());
```

```
        System.out.print("Enter Last Name: ");
```

```
user.setLastName(sc.nextLine());
```

```
boolean validUser = false;
```

```
while (!validUser) {
```

```
System.out.print("Enter Username: ");
```

```
user.setUsername(sc.nextLine());
```

```
if (user.checkUserName()) {
```

```
System.out.println("Username successfully captured.");
```

```
validUser = true;
```

```
} else {
```

```
System.out.println("Username is not correctly formatted; please ensure that your username  
contains an underscore and is no more than five characters in length.");
```

```
}
```

```
}
```

```
boolean validPass = false;
```

```
while (!validPass) {
```

```
System.out.print("Enter Password: ");
```

```
user.setPassword(sc.nextLine());
```

```
if (user.checkPasswordComplexity()) {
```

```
System.out.println("Password successfully captured.");
```

```
validPass = true;
```

```
} else {
```

```
System.out.println("Password is not correctly formatted; please ensure that the password  
contains at least eight characters, a capital letter, a number, and a special character.");
```

```
}
```

```
}
```

```
boolean validCell = false;
```

```
while (!validCell) {
```

```
System.out.print("Enter Cell Number: ");
```

```
user.setCellPhoneNumber(sc.nextLine());
```

```
if (user.checkCellPhoneNumber()) {
```

```
System.out.println("Cell phone number successfully added.");
```

```
validCell = true;
```

```
} else {
```

```
System.out.println("Cell number is incorrectly formatted or does not contain an international  
code; please correct the number and try again.");
```

```
}
```

```
}
```

```
System.out.println(user.registerUser());
```

```
System.out.println("\nLogin");
```

```
boolean isLogged = false;
```

```
while (!isLoggedIn) {  
  
    System.out.print("Enter Username: ");  
  
    String ulInput = sc.nextLine();  
  
  
    System.out.print("Enter Password: ");  
  
    String plInput = sc.nextLine();  
  
  
    isLoggedIn = user.loginUser(ulInput, plInput);  
  
    System.out.println(user.returnLoginStatus(isLoggedIn));  
  
}  
  
  
sc.close();  
  
}  
  
}
```

LoginTest.java

```
// Unit tests for authentication criteria.
```

```
import org.junit.jupiter.api.BeforeEach;
```

```
import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
public class LoginTest {
```

```
    private Login testUser;
```

```
    @BeforeEach
```

```
    public void setUp() {
```

```
        testUser = new Login("den_1", "38Baby!#", "Denau", "Hermanus", "+27769856209");
```

```
    }
```

```
    @Test
```

```
    public void testUsernameIncorrectFormatMessage() {
```

```
testUser.setUsername("denau!!!!");
```

```
String expected = "Username is not correctly formatted; please ensure that your username  
contains an underscore and is no more than five characters in length.";
```

```
assertEquals(expected, testUser.registerUser());
```

```
}
```

```
@Test
```

```
public void testPasswordIncorrectFormatMessage() {
```

```
testUser.setPassword("password");
```

```
String expected = "Password is not correctly formatted; please ensure that the password  
contains at least eight characters, a capital letter, a number, and a special character.";
```

```
assertEquals(expected, testUser.registerUser());
```

```
}
```

```
@Test
```

```
public void testCellPhoneIncorrectFormatMessage() {
```

```
testUser.setCellPhoneNumber("0769856209");
```

```
String expected = "Cell number is incorrectly formatted or does not contain an international code;  
please correct the number and try again.";
```

```
assertEquals(expected, testUser.registerUser());
```

```
}
```

```
@Test
```

```
public void testReturnLoginStatusSuccessMessage() {
```

```
String expected = "Welcome Denau, Hermanus it is great to see you again.";

assertEquals(expected, testUser.returnLoginStatus(true));

}
```

@Test

```
public void testReturnLoginStatusFailureMessage() {

String expected = "Username or password incorrect, please try again.";

assertEquals(expected, testUser.returnLoginStatus(false));

}
```

@Test

```
public void testUsernameCorrectlyFormatted() {

testUser.setUsername("den_1");

assertTrue(testUser.checkUserName());

}
```

@Test

```
public void testUsernameIncorrectlyFormatted() {

testUser.setUsername("denau!!!!!!");

assertFalse(testUser.checkUserName());

}
```

@Test

```
public void testPasswordMeetsComplexity() {  
  
    testUser.setPassword("38Baby!#");  
  
    assertTrue(testUser.checkPasswordComplexity());  
  
}
```

@Test

```
public void testPasswordDoesNotMeetComplexity() {  
  
    testUser.setPassword("password");  
  
    assertFalse(testUser.checkPasswordComplexity());  
  
}
```

@Test

```
public void testCellPhoneNumberCorrectlyFormatted() {  
  
    testUser.setCellPhoneNumber("+27769856209");  
  
    assertTrue(testUser.checkCellPhoneNumber());  
  
}
```

@Test

```
public void testCellPhoneNumberIncorrectlyFormatted() {  
  
    testUser.setCellPhoneNumber("0769856209");  
  
    assertFalse(testUser.checkCellPhoneNumber());  
  
}
```

```
}
```

```
@Test
```

```
public void testLoginSuccessful() {
```

```
    assertTrue(testUser.loginUser("den_1", "38Baby!#"));
```

```
}
```

```
@Test
```

```
public void testLoginFailed() {
```

```
    assertFalse(testUser.loginUser("den_1", "wrongPassword"));
```

```
}
```

```
}
```